

Neural Networks and Deep Learning Lab 7

Pakalapati Naga Srinivasa Varma
MC23BT004

March 9, 2026

1 Objective

The objectives of this experiment are:

- To understand the architecture and functioning of Long Short-Term Memory (LSTM) networks and analyze how LSTMs mitigate the vanishing gradient problem present in vanilla Recurrent Neural Networks (RNNs).
- To study the gating mechanisms of LSTM including the forget gate, input gate, and output gate and compare the performance of LSTM and vanilla RNN models on sequential data like sentimental analysis and multi-step time series forecasting.

2 Hypothesis

- LSTM networks should perform better than vanilla RNNs when learning long sequential dependencies as the gating mechanisms in LSTMs should allow gradients to propagate over longer time spans without vanishing.
- In time-series forecasting tasks, LSTMs should produce more stable predictions for longer prediction horizons compared to RNNs.

3 Experimental Description

3.1 Sentiment Analysis Using LSTM

The IMDB Movie Review dataset was used for binary sentiment classification.

Preprocessing steps:

- Text reviews were converted into numerical sequences and the vocabulary size was limited to the most frequent words.
- Special tokens like <PAD> for sequence padding and <UNK> for unknown words were used. All sequences were padded to a fixed length and the dataset was split into training and testing sets.

Model Architecture

- An **embedding layer** was used to convert each word index into a dense vector representation, allowing the model to learn semantic relationships between words.
- The embeddings were then processed by an **LSTM layer**, which captures contextual dependencies in the sequential text data.
- **Dropout regularization** was applied within the LSTM layer to reduce overfitting and improve generalization.
- The output from the LSTM layer was passed to a **fully connected dense layer** with a sigmoid activation function to perform binary sentiment classification.

Training Setup

- The model was trained using the **Binary Cross-Entropy loss function**, which is appropriate for binary classification tasks.
- The **Adam optimizer** was used for training because of its adaptive learning rate and efficient convergence.
- **Early stopping** was employed to prevent overfitting by stopping the training process when the validation performance stopped improving.

3.2 Training Curves

Figure 1 shows the training and validation loss and accuracy curves for the sentiment classification model.

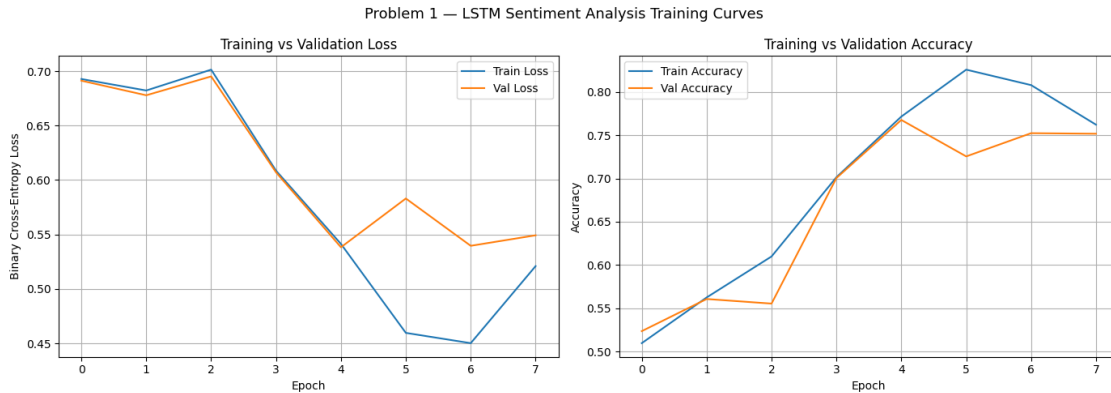


Figure 1: Training vs Validation Loss and Accuracy for the LSTM Sentiment Classifier

3.3 Confusion Matrix

The confusion matrix for the sentiment classifier is shown in Figure 2.

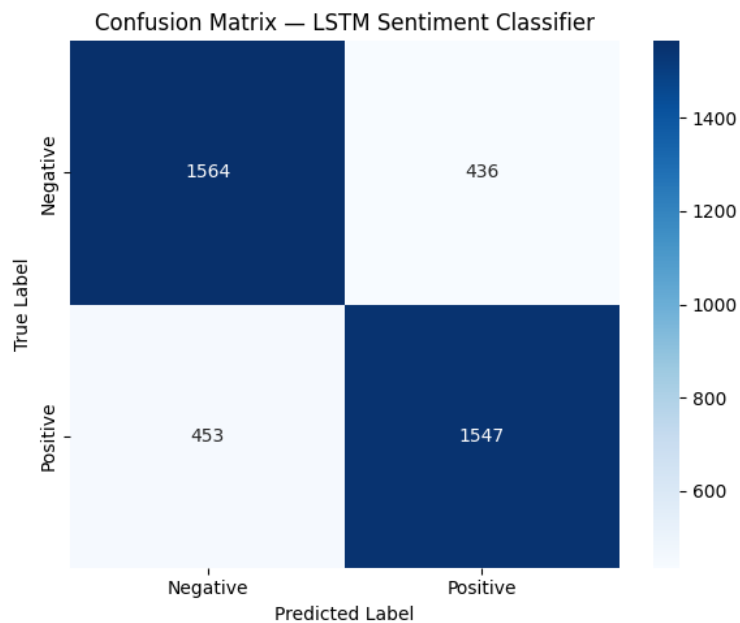


Figure 2: Confusion Matrix for the LSTM Sentiment Classifier

4 Time-Series Forecasting Experiment

4.1 Dataset Generation

A synthetic time-series signal was generated using:

$$y(t) = \sin(t) + 0.5 \cos(2t) + \text{noise}$$

Gaussian noise was added to simulate real-world measurement noise.

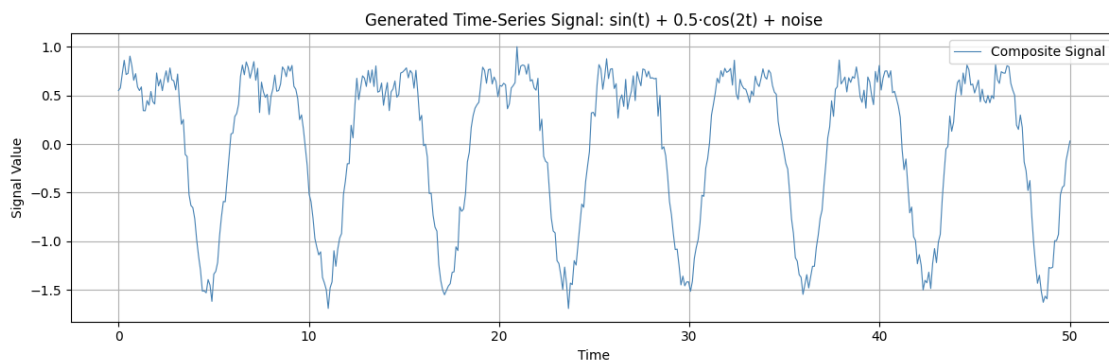


Figure 3: Generated Composite Time-Series Signal

4.2 Sliding Window Dataset

To convert the time-series signal into supervised learning samples, a sliding window approach was used. In this method, the model observes a fixed number of past time steps and predicts the future values of the signal.

- Input sequence length: 50 time steps
- Prediction horizon: 5 time steps

Thus, each training sample consists of the previous 50 time steps as input and the next 5 time steps as prediction targets. This formulation allows the models to learn temporal dependencies between past observations and future values.

4.3 Model Comparison

Two recurrent models were trained on the generated time-series dataset in order to evaluate their ability to learn sequential patterns:

- Vanilla Recurrent Neural Network (RNN)
- Long Short-Term Memory Network (LSTM)

Both models were trained using the Mean Squared Error (MSE) loss function, which is suitable for continuous-value prediction problems such as time-series forecasting.

4.4 Training Loss Comparison

Figure 4 shows the training loss of both models across epochs.



Figure 4: Training Loss Comparison Between RNN and LSTM

From the figure, it can be observed that the LSTM model converges more smoothly compared to the vanilla RNN. The additional gating mechanisms in LSTM help stabilize training and improve the model's ability to capture longer temporal dependencies.

4.5 Prediction Comparison

Figure 5 shows the comparison between the predictions produced by the RNN and LSTM models.

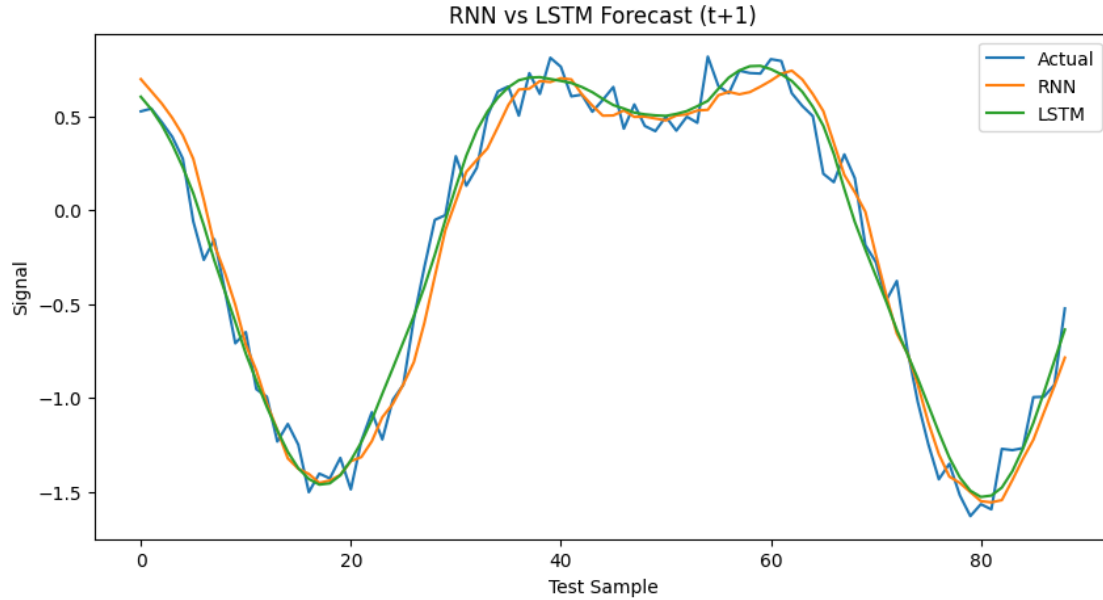


Figure 5: Comparison of RNN and LSTM Predictions for Time-Series Forecasting

Both models are able to capture the general waveform of the signal. However, the LSTM model produces smoother predictions and maintains closer alignment with the true signal over time.

4.6 Multi-Step Forecast Example

An example of a multi-step forecast for a single test sample is shown in Figure 6.

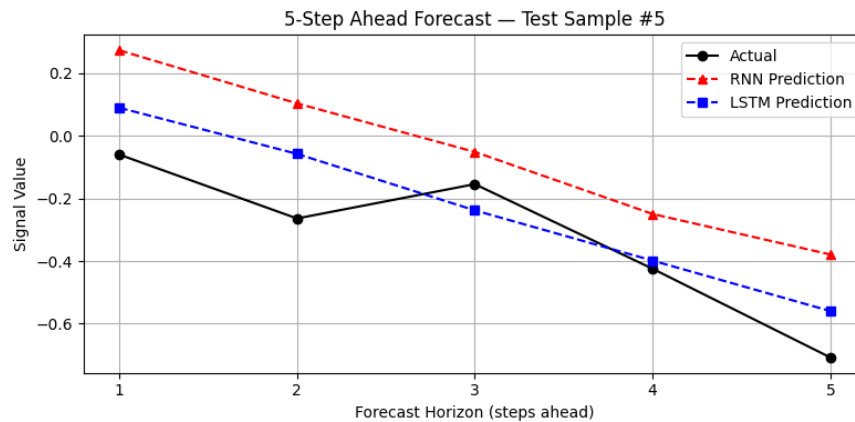


Figure 6: Example of 5-Step Ahead Forecast for a Test Sample

This figure illustrates how both models attempt to predict the next five time steps after observing the past sequence. While both models follow the overall trend of the signal, the LSTM predictions remain closer to the actual future values as the forecast horizon increases.

5 Observations

The following observations were made from the experiments:

- During sentiment analysis training, the loss decreased steadily while validation accuracy stabilized around 75–80%, indicating reasonable generalization.
- Early stopping prevented further overfitting once validation performance stopped improving.
- In the time-series forecasting experiment, both RNN and LSTM were able to capture the general waveform of the signal.
- However, LSTM predictions were smoother and remained closer to the true signal compared to the vanilla RNN.
- The RNN model showed slightly larger deviations as the prediction horizon increased.

6 Conclusion

- The LSTM cell state C_t acts as a nearly linear memory path (often called a constant error carousel), allowing gradients to propagate across many time steps without vanishing. This enables LSTMs to learn long-term dependencies more effectively than vanilla RNNs.
- The forget gate controls how much information from previous time steps should be retained. By multiplying the previous cell state with a value between 0 and 1, it selectively removes irrelevant information and prevents uncontrolled accumulation of past memory.
- LSTMs require more parameters and computation than vanilla RNNs because they maintain multiple gates (forget, input, output) along with a candidate cell state. Although this increases model complexity, it significantly improves stability when learning long sequential patterns.
- LSTMs are useful in several real-world sequential tasks. In speech recognition they model temporal patterns in phoneme sequences; in financial forecasting they learn trends and autocorrelations in price series; in biomedical signal processing they detect patterns in signals such as ECG or EEG; and in sensor data analysis they capture long-term degradation patterns for predictive maintenance.